

N = 8 # Size of the chessboard

def print_solution(board):

"""Function to print the solution."""

for row in board:

print(" ".join("Q" if cell else "." for cell in row))

print("\n")

def is_safe(board, row, col):

"""Check if placing a queen at (row, col) is safe."""

Check the same column

for i in range(row):

if board[i][col] == 1:

return False

Check upper left diagonal

for i, j in zip(range(row, -1, -1), range(col, -1, -1)):

if board[i][j] == 1:

return False

Check upper right diagonal

for i, j in zip(range(row, -1, -1), range(col, N)):

if board[i][j] == 1:

return False

return True # Safe to place the queen

def solve_n_queens(board, row):

"""Recursive function to solve N-Queens problem."""

```

if row >= N: # All queens are placed
    print_solution(board)
    return True

for col in range(N):
    if is_safe(board, row, col):
        board[row][col] = 1 # Place queen

        if solve_n_queens(board, row + 1): # Recur for the next row
            return True

        board[row][col] = 0 # Backtrack and remove the queen

return False # No valid position found

```

```

def solve():
    """Initialize the board and solve the problem."""
    board = [[0] * N for _ in range(N)] # Create an 8x8 board filled with 0
    if not solve_n_queens(board, 0): # Start solving from the first row
        print("No solution exists")

```

```

# Run the solver

```

```

solve()

```